

# Helix Thready — Secrets & Config

## Table of Contents

- 1. Non-negotiable rules
- 2. Secret sources & load precedence
- 3. Secrets flow diagram
- 4. File conventions & permissions
- 5. Local git-hook enforcement (no server CI)
- 6. Leak-audit & rotation
- 7. Secrets at rest & in backups
- 8. Mandatory config keys (incl. gap enforcement)
- 9. Verified vs assumed
- 10. Open items

| Rev | Date       | Author                    | Change                                                                                 |
|-----|------------|---------------------------|----------------------------------------------------------------------------------------|
| 1   | 2026-07-21 | swarm (deployment)        | Initial secret sources, load precedence, chmod, leak-audit, local git-hook enforcement |
| 2   | 2026-07-21 | swarm (deployment review) | Split the secrets-flow prose into multiple paragraphs                                  |

This document specifies how Helix Thready handles secrets and runtime configuration: **runtime-load only** from gitignored `.env` / `api_keys.sh`, `chmod 600/700`, never logged, never committed to a public repo, with **local git-hook enforcement** in place of the forbidden server-side CI ( `§14.4` , Q39, `[CONSTITUTION §11.4.10 / §11.4.156]` ).

Diagram source: sibling under `diagrams/` . Rendered PNG/SVG exported via Docs Chain (§11.4.65).

## Table of Contents

- 1. Non-negotiable rules
- 2. Secret sources & load precedence
- 3. Secrets flow diagram
- 4. File conventions & permissions
- 5. Local git-hook enforcement (no server CI)
- 6. Leak-audit & rotation

7. Secrets at rest & in backups
  8. Mandatory config keys (incl. gap enforcement)
  9. Verified vs assumed
  10. Open items
- 

## 1. Non-negotiable rules

---

From `[CONSTITUTION §11.4.10]` and `§14.4` — these are hard constraints, not preferences:

- **No credentials in any public repo**, ever — not source, not config, not docs, not logs.
- **Runtime-load only** — secrets are read from gitignored `.env` / `secrets` files *at process start*, never baked into an image or a tracked file.
- **chmod 600 files, 700 dirs** — secret files are owner-only.
- **SKIP-if-missing** — a missing credential makes the dependent feature *skip cleanly*, never crash loudly with the secret in the message, and never fall back to a hardcoded value.
- **Never logged** — secrets are redacted from all structured logs (`observability`/logrus).
- Sensitive materials (the private submodule, credentials, the gap register) live in the **private** repo (`helix_thready_private`), never `docs/public`.

## 2. Secret sources & load precedence

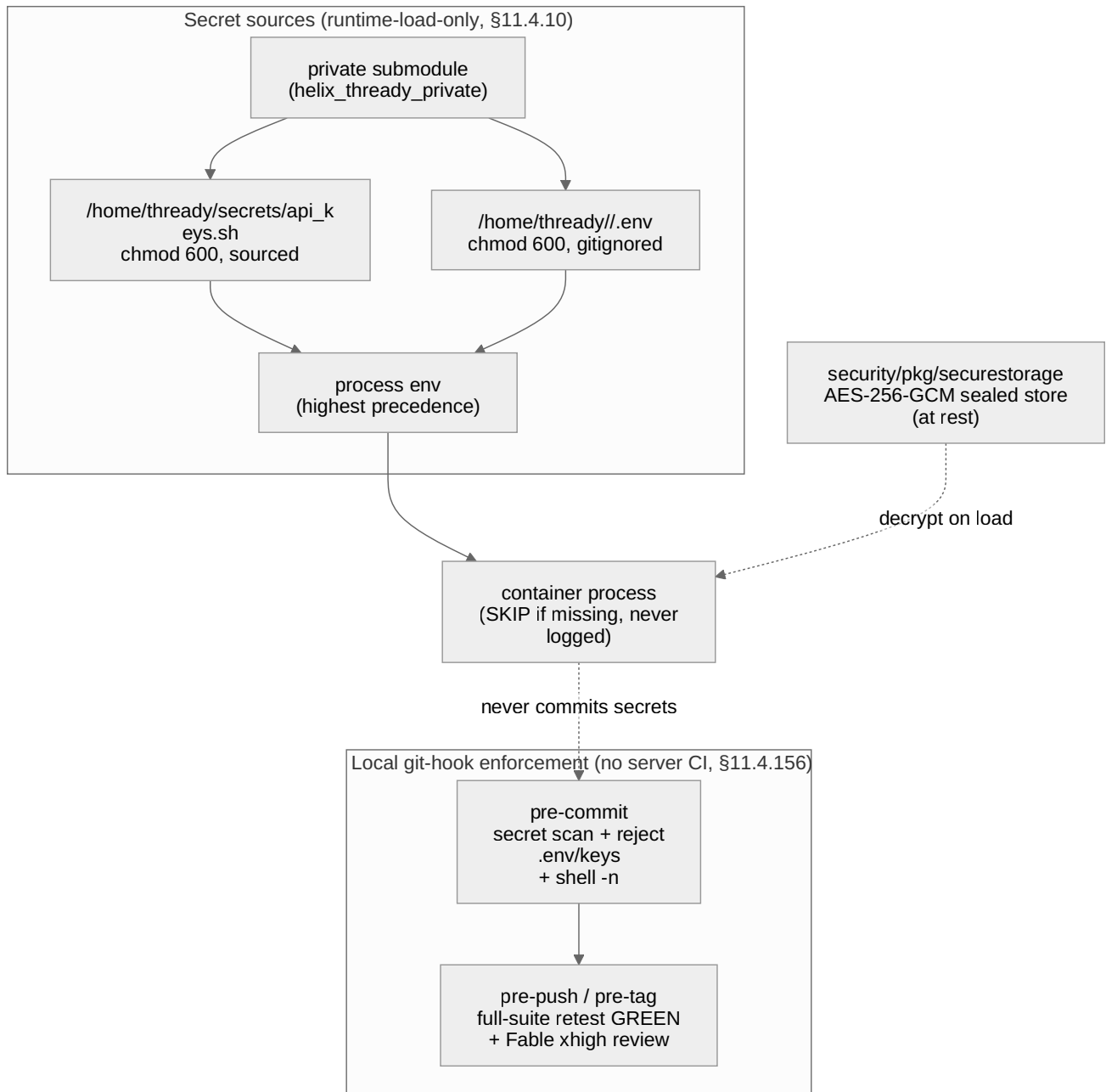
---

Three sources (`Appendix B` of the final request), highest precedence first:

1. **Process environment** — variables already exported into the container/process win. This is how the deploy script and systemd units inject per-run values.
2. `/home/thready/<env>/.env` — the per-environment dotenv, `chmod 600`, gitignored. Compose reads it via `env_file:`; the deploy script `source`s it.
3. `/home/thready/secrets/api_keys.sh` — a host-home shell file that `export`s API keys (LLM providers, messenger tokens, DNS-01 secrets), `chmod 600`, sourced at runtime.

The ultimate source of the values is the **private submodule**; they are copied onto the host securely at provisioning and refreshed on rotation. Nothing is fetched from a public location.

### 3. Secrets flow diagram



**Explanation (for readers/models that cannot see the diagram).** The authoritative secret values originate in the **private submodule**, which is visible only to the operator and the CLI agent. From there they are materialized on the host as the per-environment `.env` file and the host-home `api_keys.sh`, both `chmod 600` and gitignored. At process start, `api_keys.sh` is sourced and the `.env` is loaded, populating the **process environment**; anything already in the environment (set by the deploy script or a systemd unit) takes precedence over the files.

The container process reads its config from that environment and, crucially, **skips cleanly** if a required secret is missing rather than crashing with the secret in the error, and **never logs** the value. Secrets that

must persist at rest (e.g. sealed tokens) go through `security/pkg/securestorage`, which seals them with AES-256-GCM and decrypts only on load.

The bottom band is the enforcement that a server-side CI would normally provide but which is **forbidden** here: a local **pre-commit** hook scans every staged change for secrets and rejects any attempt to commit a `.env` or key file (plus it shell-lints scripts), and a **pre-push / pre-tag** hook runs the full test suite to GREEN and the independent AI review before a release tag can be cut. Together these keep secrets out of history without any remote automation.

## 4. File conventions & permissions

```
/home/thready/
├─ dev/.env          chmod 600  THREADY_DEV_*  secrets
├─ sta/.env          chmod 600  THREADY_STA_*  secrets
├─ prod/.env         chmod 600  THREADY_PROD_* secrets
├─ secrets/
│   └─ api_keys.sh   chmod 600  export HELIX_LLM_API_KEY=... HERALD_TELEGRAM_* ... LOOPIA_*
└─ <env>/config/
    └─ lets_encrypt.conf chmod 600  (no DNS secrets inside – those come from api_keys.sh)
```

`.gitignore` in the repo already excludes `.env`, `*.env`, `secrets/`, `api_keys.sh`, cert/key files. The `container-topology` `DefaultHelixServices()` **dev-only** fallback credentials are **never** used in prod — every credential is injected from these files (`mustEnv` fails fast if one is missing, per `container-topology.md` §10).

Example `.env` (values are placeholders; real ones come from the private repo):

```
# /home/thready/prod/.env (chmod 600, gitignored)
THREADY_ENV=prod
THREADY_PORT_PREFIX=62
THREADY_PUBLIC_HOST=thready.hxd3v.com
THREADY_PG_USER=thready_prod
THREADY_PG_PASSWORD=__from_private_repo__
THREADY_PG_DSN=postgres://thready_prod:__from_private_repo__@thready-postgres:5432/thready
THREADY_NATS_URL=nats://thready-nats:4222
THREADY_MINIO_ENDPOINT=http://thready-minio:9000
HELIX_EMBEDDING_PROVIDER=llama          # GAP #1 – mandatory
HELIX_LLM_BASE_URL=http://gpu-node.helix.lan:8080
THREADY_LOG_LEVEL=info
```

## 5. Local git-hook enforcement (no server CI)

[GAP: #12 CI-equivalent] [CONSTITUTION §11.4.156] — server-side CI (GitHub Actions/GitLab CI/Jenkins) is **forbidden**. The equivalents run as **local git-hooks** (§11.4.75) installed by a repo `make hooks` target.

**pre-commit** — fast, blocks bad commits:

```
#!/usr/bin/env bash
# .github/hooks/pre-commit
set -euo pipefail

# 1. Reject staging any secret file outright.
if git diff --cached --name-only | grep -E '^(^|/)(\.env$|.*\.env$|api_keys\.sh$|.*\.pem$|
    secrets/)' ; then
    echo "FATAL: refusing to commit a secret file (§11.4.10)"; exit 1
fi

# 2. Content secret-scan the staged diff (gitleaks-style patterns: tokens, keys, DSNs with
    creds).
git diff --cached -U0 | scripts/secret-scan.sh || { echo "FATAL: secret-like content detected";
    exit 1; }

# 3. Shell scripts must parse (bash -n / sh -n) – matches lets_encrypt's own test gate.
for f in $(git diff --cached --name-only | grep -E '\.sh$'); do bash -n "$f"; done
```

**pre-push** / **pre-tag** — the release gate:

```
#!/usr/bin/env bash
# .github/hooks/pre-push
set -euo pipefail

make test-all          # full-suite retest GREEN (§11.4.40) – the 15 mandated test types
scripts/ai-review.sh --model fable --effort xhigh
                        # independent AI review (§11.4.209), iterate to GO
```

- A **THREADY-`<ver>`** tag is only cut after the full suite is GREEN and the AI review returns GO (§11.4.134); then the release fans out to **all four upstreams** (§2.1).
- **Defense in depth:** the deploy script itself re-checks `.env`, `chmod 600`, secret presence, and `HELIX_EMBEDDING_PROVIDER=llama` at run time — so a host is refused even if a hook was bypassed locally.

## 6. Leak-audit & rotation

[CONSTITUTION §11.4.10] requires leak-audit + rotate-on-leak:

- **Continuous scan** — `security/pkg/scanner` (Snyk/SonarQube adapters) + the `secret-scan.sh` patterns run over the repo history periodically; any hit is a P0 incident.
- **Rotate-on-leak runbook:**
  1. Revoke the exposed credential at its source (LLM provider console, messenger, DB `ALTER ROLE ... PASSWORD`, or `lets_encrypt revoke.sh` + `rotate.sh` for a cert/key).
  2. Regenerate and place the new value in the private repo → refresh `.env` / `api_keys.sh` on the host (`chmod 600`).
  3. Redeploy the affected env (`deploy-and-rollback.md`); the health gate confirms the new credential works.

4. Purge the leaked value from git history if it ever reached a tracked file (BFG/ `filter-repo` ), then force-refresh all four upstreams **once** (the only sanctioned history rewrite).

- **Firestore signing keys** ( §14.4 ) — dynamically generated, **owner-only**, stored in the private repo; never on the host beyond the moment of use.

## 7. Secrets at rest & in backups

- **At rest** — secrets that must be stored (sealed tokens, OAuth refresh tokens) use `security/pkg/securestorage` AES-256-GCM with an Argon2id-derived key (Q38). The DB itself is encrypted-at-rest per the security posture; the `.env` on disk is `chmod 600` (host-FS protected).
- **In backups** — `.env` / `api_keys.sh` are included in the `config backup` **only encrypted**, never plaintext, and only to the private secondary — never a public location.
- **Semantically-searchable secrets** ( §3.6 ) — credentials that must be *findable* are embedded over a **redacted/tokenized** representation so the vector index can locate them without ever storing the raw value ( `[GAP: security §7.1]` improvement).

## 8. Mandatory config keys (incl. gap enforcement)

| Key                                         | Where                                    | Enforced by                                                | Gap                                          |
|---------------------------------------------|------------------------------------------|------------------------------------------------------------|----------------------------------------------|
| <code>HELIX_EMBEDDING_PROVIDER=llama</code> | every env <code>.env</code>              | deploy pre-flight + semsearch health probe                 | <code>[GAP: #1]</code> fail-if-hash-embedder |
| <code>.env</code> is <code>chmod 600</code> | host FS                                  | deploy pre-flight <code>stat -c %a</code>                  | <code>[GAP: #12]</code>                      |
| no secret file in git                       | repo                                     | <code>pre-commit</code> hook                               | <code>[CONSTITUTION §11.4.10]</code>         |
| <code>HELIX_LLM_BASE_URL</code> reachable   | env                                      | boot Phase-1 discovery (Required)                          | <code>[GAP: #1]</code> external LLM          |
| DNS-01 secrets from env only                | <code>api_keys.sh</code>                 | <code>lets_encrypt</code> reads env, config has no secrets | <code>[CONSTITUTION §11.4.10]</code>         |
| DB creds injected (no literal)              | <code>.env</code> → <code>mustEnv</code> | <code>mustEnv</code> fails fast                            | <code>[GAP: #12 anti-bluff]</code>           |

`[GAP: #1 HelixLLM HashEmbedder]` is enforced in **two** independent places — the deploy pre-flight refuses to proceed if `HELIX_EMBEDDING_PROVIDER != llama` , and the `thready-semsearch` readiness probe fails if the active provider is the non-semantic hash embedder — so no environment

can ever silently serve garbage-relevance semantic search. This is the deployment-side closure of the P0 gap.

## 9. Verified vs assumed

- **VERIFIED:** the three secret sources + runtime-load-only + SKIP-if-missing + never-logged ( `Appendix B`, `$11.4.10` ); no-server-CI + local-hooks (Q21, `$11.4.156/75` ); `security/pkg/securestorage` AES-256-GCM ( `$2.1`, Q38); `lets_encrypt` reads DNS-01 secrets from the environment (module README); private-repo-only sensitive materials ( `$1.1` ).
- **ASSUMED / [DEFAULT – adjustable]:** the `secret-scan.sh` / `ai-review.sh` wrapper names and the exact `.env` key names for Thready's *own* services; the BFG-vs-filter-repo history-purge tool.

## 10. Open items

- `[OPEN: sops-age]` — the final request notes SOPS/age *may* be added over the file-based secrets if desired (Q39); not required for MVP, tracked as an optional hardening.
- `[OPEN: secret-scan-tool]` — whether `secret-scan.sh` wraps gitleaks, trufflehog, or the `security/pkg/scanner` patterns is an implementation choice for the *development* area.

Made with love ♥ by Helix Development.